

Vulnerability Report

Report ID: VULN-001
Date: July 3, 2026
Reported By: Malik (Security Task Force – Day 1)
Target Application: OWASP Juice Shop (local sandbox, localhost:3000)
Environment: Docker container bkimminich/juice-shop , deployed locally via Docker Desktop on Windows 11 (WSL2 backend)

1. Summary

Field	Value
Title	Reflected/DOM-based Cross-Site Scripting (XSS) in Product Search
Severity	High
CVSS 3.1 Score	6.1 (Medium-High)
CVSS Vector	AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N
CWE Classification	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site

Field	Value
	Scripting')
OWASP Top 10 Category	A03:2021 – Injection
Affected Component	Product Search feature (/#/search?q=)

2. Description

The product search functionality on the OWASP Juice Shop application does not properly sanitize or encode user-supplied input before rendering it into the DOM. This allows an attacker to inject arbitrary HTML/JavaScript via the search query parameter, resulting in a Cross-Site Scripting (XSS) vulnerability.

While a direct `<script>` tag injection was blocked (browsers do not execute `<script>` tags inserted via `innerHTML`), an alternate vector using an `<img>` tag with an `onerror` event handler successfully bypassed this restriction and executed arbitrary JavaScript in the browser context.

3. Steps to Reproduce

1. Navigate to `http://localhost:3000`

2. Click the search icon in the top navigation bar
3. Enter the following payload into the search field:

```
<img src=x onerror=alert('XSS')>
```

4. Press **Enter** to submit the search query
5. Observe that a JavaScript `alert()` dialog box fires immediately, displaying "XSS"
6. Confirmed via Chrome DevTools → Elements tab that the raw `` tag is present, unescaped, in the rendered DOM

Note: A plain `<script>alert('XSS')</script>` payload was attempted first and did not execute (expected browser behavior — scripts inserted via `innerHTML` are not executed by the DOM parser). This confirms the vulnerability is exploitable via event-handler-based payloads, which is a common and equally serious variant of XSS.

4. Expected vs. Actual Behavior

	Behavior
Expected	User input in the search field should be sanitized/HTML-encoded before being rendered, so that any HTML tags are displayed as literal text (e.g., <code>&lt;img</code>

	Behavior
	<code>src=x onerror=...&gt;)</code> rather than executed.
Actual	The raw HTML payload is rendered directly into the DOM without encoding, allowing the <code>onerror</code> event handler to execute arbitrary JavaScript in the victim's browser session.

5. Evidence

- **Screenshot 1:** Payload entered into the search input field (pre-submission)
- **Screenshot 2:** JavaScript alert dialog triggered post-submission, with vulnerable URL visible in address bar
- **Screenshot 3:** Docker Desktop confirming the Juice Shop sandbox container running on port 3000:3000 (proof of isolated local testing environment)
- **Screenshot 4:** Chrome DevTools Elements panel showing the unescaped `` tag injected directly into the DOM

(See attached image files in repository `/evidence` folder)

6. Impact

An attacker could exploit this vulnerability to:

- Steal session cookies or authentication tokens (session hijacking)
- Perform actions on behalf of the victim (CSRF-like abuse)
- Redirect users to malicious/phishing sites
- Deface the application UI
- Deliver further malware via injected scripts

Since this is a DOM-based/reflected XSS, exploitation typically requires the victim to click a crafted malicious link (e.g., via phishing), making **User Interaction: Required** in the CVSS vector above.

7. Remediation

Recommended fixes:

1. **Output encoding:** HTML-encode all user-supplied input before rendering it into the DOM. Any framework-level auto-escaping (e.g., Angular's built-in sanitization) should not be bypassed via `innerHTML`, `bypassSecurityTrustHtml()`, or similar unsafe APIs.
2. **Input validation:** Apply a strict allow-list of acceptable characters for search queries where

feasible (though encoding is the primary defense – validation alone is insufficient).

3. **Content Security Policy (CSP):** Implement a strict CSP header (e.g., `script-src 'self'`) to prevent inline script/event-handler execution even if a payload is injected.

Example remediation code (Angular/TypeScript context):

```
// VULNERABLE: renders raw HTML
this.searchResultElement.innerHTML =
userSearchQuery;

// SAFE: use Angular's built-in text binding
(auto-escapes)
// In template:
// <div>{{ userSearchQuery }}</div>

// If HTML rendering is genuinely required,
sanitize explicitly:
import { DomSanitizer } from
'@angular/platform-browser';

constructor(private sanitizer: DomSanitizer)
{}

getSafeHtml(input: string) {
  // Strips dangerous tags/attributes instead
  of trusting blindly
  return
  this.sanitizer.sanitize(SecurityContext.HTML,
```

```
input);  
}
```

```
// Server-side defense-in-depth  
(Node.js/Express example)  
const sanitizeHtml = require('sanitize-  
html');  
  
app.get('/search', (req, res) => {  
  const safeQuery = sanitizeHtml(req.query.q,  
  {  
    allowedTags: [],  
    allowedAttributes: {}  
  });  
  // proceed with safeQuery instead of raw  
  req.query.q  
});
```

8. References

- OWASP Top 10 2021 – A03:2021 Injection:
https://owasp.org/Top10/A03_2021-Injection/
 - CWE-79:
<https://cwe.mitre.org/data/definitions/79.html>
 - PortSwigger Web Security Academy – XSS:
<https://portswigger.net/web-security/cross-site-scripting>
-

9. Ethical & Scope Notes

- All testing was performed manually against a **locally-deployed, isolated Docker sandbox** (OWASP Juice Shop) with no public server interaction.
- No automated scanners were used to discover this finding; exploitation was manually reproduced to confirm the underlying vulnerability mechanism, per assignment guidelines.
- This finding is intended solely for educational/training purposes as part of the Cyber Security Task Force Day 1 assignment.